

Introducing Agile into a Software Development Capstone Project

Cyril Coupal and Kelvin Boechler

Computer Systems Technology Program, Saskatchewan Institute of Applied Science and Technology
Saskatoon, Saskatchewan, S7K 3R5

coupal@siast.sk.ca
boechlerk@siast.sk.ca

Abstract.

Conveying principles of software analysis, design and implementation in a classroom setting is problematic. When the course involves actual hands on development with clients drawn from industry, the challenges are magnified. This paper discusses the experiences and observations of a set of 10-month independent external projects undertaken by final year students in the Computer Systems Technology Program using Agile for the first time. We compare situations and observations of projects developed following an Agile [1] approach with XP Programming [2], to our previous projects developed in a traditional approach. Based on these observations, an Agile approach seems to support learning, provide a valuable practical experience and produce useable software within an academic environment.

1. The Capstone Project Course

The two-year diploma Computer Systems Technology Program (CST) at the Saskatchewan Institute of Applied Science and Technology (SIASST) Kelsey Campus has been in existence for approximately 20 years. During this time, one course has set the program apart from similar programs at sister campuses. The capstone project course focuses on team analysis, design and implementation of an Information Technology (IT) based application for clients drawn from industry or from some other program within SIASST.

The course was originally developed to help the student obtain an experience-based appreciation for their knowledge development and acquired soft skills. It was placed in the later portion of their education to utilize as much of their newly acquired knowledge as possible in actual development.

The two one-term courses associated with the capstone project are *COSA280 Rapid Application Development*, and *COSA290 Visual Application Development* [3]. Both

names reflect the leading edge approach to software development in place at the time the courses were first introduced. *COSA280* was introduced to expose the student to the concepts of developing prototypes in a Rapid Application Development (RAD) environment. The beginnings of the course preceded, by some years, widespread integrated development environments (IDEs) based on graphical user interfaces (GUIs). *COSA290* was introduced to present students with visual development environments as they became more commonplace. With the introduction of object-oriented systems and the evolving sophistication of IDEs to provide drag-and-drop development, the formal approach to rapid application development through prototyping in the capstone project was largely abandoned. The names of the courses have remained, however.

1.1. Nature of the Project Class

The second year CST *COSA280-290* classes span two terms from the middle of August until the end of April. There are two instructors assigned to the courses for the full year. During this time, up to 50 students in groups of 5...8 members, work approximately 4 hours per week during assigned class time. Each group is assigned to a specific project with an external client. These projects are completely independent of each other, drawn from businesses in industry.

The scope of the project tends to be limited due to the short, fixed duration of the courses and the limited size and expertise of the groups. Often the client is exploring some idea and their own IT department does not have the time to pursue the idea, or the client is looking for a solution to a small problem and does not want to spend funds on development. There is no cost to our client organizations to take part in this project, other than a time commitment.

The group interacts with their client to analyze, design and implement a working computer-based application to fill some need of the client. The client understands that

the application is expected to require significant refactoring to provide what the client requests in production form after the project terminates. Thus a partially functioning application can still be useful to the client as it helps determine needs and potential solutions even if not all of the required functionality is implemented within the course time.

The mark reflects the degree to which the needs of the client were identified and how well the implementation demonstrates the likelihood of an IT solution being possible and effective in addressing those needs.

1.2. Former Project Development Process

The historic approach to software development within the two classes utilized the waterfall model [4] of software development. During the fall term, students were expected to interact with the client and develop an analysis model. Although the time for completion of this model was flexible, it had to be formalized by the end of the term. Students were encouraged to be into design model development before the end of the first term.

The second term usually consisted of a push to complete the design model by midterm break and then implement the model in a development environment by the end of the second term. Formal client sign-off on documentation and models was the milestone allowing students to advance to the next stage of development. Coding was largely forbidden except where exploration of some concept or idea was necessary. The models presented to the client have evolved from early documentation with Entity-Relationship [5] data models, algorithms and structured code, to Unified Modeling Language (UML) [6] models being used currently.

1.3. Former Project Successes and Failures

In the early years of the course, projects were deemed successful, as a satisfactory percentage of the client requirements were usually implemented and useable. Deliverables to the client included all documentation, models and source code. In addition, any database tables developed for the application were given to the client. If the final product was incomplete or unstable, having complete and accurate models was beneficial to the client. The client could then approach a commercial developer with the models and student software for development of the actual production version. Often this was unnecessary because the project was useful enough that the client did not need to incur the cost of developing an actual production version, even though the student software was not production quality. Students seemed proud of their achievement and were happy to demonstrate their working product. The functioning application formed a significant portion of their evaluation.

As later projects became more complex, less and less functionality required by the final product was possible to implement. In response to this trend, student evaluation shifted more towards analysis and design models, which were more significant, and away from implemented code.

Another difficulty with these more complex projects was that the students could not estimate the complexity of the implementation during analysis and design model development. They found that what appeared fairly simple in their models would, in reality, be quite complex to implement, especially within the relatively short period of time left for implementation. As they became aware that they could not implement any significant amount of production functionality, students would become disappointed or depressed. Their focus then turned to the mark they hoped to receive rather than on the completion of the project with satisfactory software. With each year, students appeared less satisfied with the outcome of the project.

In section 2.1 we discuss the student behavior that underlined the need for change in our approach to software development within the capstone project course.

2. Rationale for Change to an Agile Approach

Any approach to software development that enhances learning while increasing productivity, from the client and student standpoints, is desirable. Due to the sophistication of today's business technology environment, requirements are generally poorly defined or unknown at the initiation of the project. Traditional approaches to software development have no longer proven ideal for development of applications [2]. A number of observations into client and student behavior pointed the project course towards an Agile/XP approach.

2.1. Observed Student Behavior

Typically, during the analysis and design phases, students would settle into roles, partitioning the work among themselves. Each person would become adept at their given task but would not participate in other tasks outside their chosen area. We had groups where a single student would do documentation for the entire 8 months and never write a line of code. This "silo effect" on division of labor was detrimental in a number of ways:

- Although the capstone project was to be a major learning experience, students would learn only their individual area of work. There was little motivation to work in an area the student found difficult. The group would discourage participating in tasks for which the student did not excel, because this would slow progress

as others helped the less skilled student develop their abilities.

- The students would become quite possessive of their own work. This would lead to a resistance to change or delete any portion of their contribution from the project. Thus, rather than change to a possibly better solution they would maintain their original solution and continue to try and make it succeed.
- Students would often work in separate rooms, by themselves, so that they could concentrate on “their” tasks.
- Because of the silo effect on tasks, we would find we were repeatedly helping different students within the same project group solve the same problem. For example, one student may have a problem manipulating dates. After solving the problem with the student, we would find that another student was having a similar date problem. We also found that if a student discovered a solution, that knowledge would not likely become known to the rest of the group or other groups.

Team cohesion would usually start to erode as time became short and it became evident the group would not complete the project. Rather than pull together, whoever was responsible for coding usually began heroic measures to attempt to complete the functionality while non-coding members polished the documentation or sat idly by since they knew nothing of the code base developed to date.

Although traditional methods were formally employed in the project class, students often exhibited agile-like behavior. We discovered students were coding functionality even though the evaluation structure did not provide marks for coding or functionality in that particular phase of the project. Although the traditional approach emphasized the importance of completing a correct model, the students and the client would both deem the application code of greater importance. The subsequent conflict between what the course required the students to produce and what they and their client wanted produced, eroded interest and morale reducing the likelihood of successfully completing the project.

As the evaluation structure evolved into one that was model-centric, student effort shifted as well. Students would often become obsessed with perfecting the models, focusing mostly on diagram layout. Given increasing complexity, significant model content was often provided by the instructors since the students had insufficient expertise or experience to identify and capture that content themselves.

During this period of development, we observed the silo effect with some group members uninterested in analysis and design, working on programming assignments from other classes, or simply surfing the web. With the end of the project months away, the students did not feel the need to work diligently on the

project: an assignment due tomorrow seemed more important than a project to be delivered in six months.

Periodically, students would be presented with information in another class that they would eventually need to complete their project. However, because implementation was left until the latter part of the project, this information would be largely forgotten and have to be relearned. Since implementation often resulted in a heroic effort, by a few, to complete the remaining functionality, this information was never assimilated properly or widely. Misunderstandings and confusion were evident in the kinds of questions that the students asked often indicating a fundamental misunderstanding of the concepts learned earlier but not previously employed in a project setting.

The project class did little to increase student interest in other classes as the other classes focused more on implementation issues.

2.2. Observed Client Behavior

A difficulty that faced most clients was to understand the models presented to them by the students during their informational meetings. Most of our clients had no background in software development. The students would attempt to explain to the client the models and the consequences of decisions, based on those models, during the first few meetings. Eventually the clients would come to understand something of the model content but never to the level where they could make informed decisions about future requirements and functionality. This resulted in a significant loss of time that could have been put to use identifying functionality.

Prototype screens became very important, as projects grew more complex, since both students and client could easily understand the graphical nature of a screen and relate it to functional requirements. The difficulty was that the screen became the focus of client/student interaction with effort being expended on screen presentation. Colors, choice and position of controls tended to dominate the effort while major design decisions were ignored until it became impossible to ignore them further.

2.3. Changing Client Expectations

Early projects were successful because they tended to be small standalone non-networked applications designed for DOS® and Microsoft® Windows 3.1 environments. Applications were usually file based, with minimal reporting. Database requirements were largely met by Dbase® [7]. Later, with the introduction of Microsoft Access® and the ability to produce Windows forms in the Access IDE, Access became the environment of choice for many project efforts.

As client expectations increased, projects became more complex. Networking became common in many business

environments and applications were no longer standalone. As the Internet became more ubiquitous, applications were expected to have web-based and email components. Often applications were expected to interact with commercial products to produce charts and formatted documents.

The traditional approach to software development still produced good models but more effort than ever before was being expended capturing and perfecting the model. As a consequence, the groups implemented less and less functionality. In our final year using traditional software development, 2003-2004, not one project achieved even 50% of agreed-to functionality by the end of the project course. Most projects achieved far less. Clients still obtained a good model and found the exercise valuable but did not have anything remotely useful, from a functional standpoint. At the end, even heroic efforts involving many out-of-class hours, by the students and instructors, did not significantly increase the useful functionality of the final deliverables.

2.4. Changing Employer Expectations

In parallel to the evolving changes in the nature of the project, there was evidence that employer expectations were changing as well. Recruitment notices made references to many skills not common only a few short years ago. The typical diploma graduate today requires knowledge of networks, Internet protocols and applications, web-based presentation of information and interfaces to shared data. The graduate requires knowledge in multiple environments and platforms. Virtually every application has a database component of some kind.

Student exposure to many different development languages and environments is desirable. Employers tend to expect our graduates to have a significant amount of hands-on experience with these languages and environments. This hands-on experience has largely been what distinguishes our graduates from those of computer science programs at university.

3. Current Class Nature

The current academic year began in August 2004. We returned from the Extreme Programming and Agile Methods – XP/Agile Universe 2004 [8] conference armed with many ideas and support material for altering the nature of the project course to be more Agile. To formally change the course content and direction within an academic institution such as ours can take up to 18 months for all approvals and calendar changes. Our first hurdle, as instructors of the course, was to implement an Agile approach and XP programming in a manner that did not conflict with published descriptions and outcomes for

the course. We thus used an “agile” approach and modified what we understood of Agile to fit our needs while remaining within the context of published course descriptions [3].

This is the first introduction of Agile and XP Programming into the program. Our second-year students had no previous exposure to the practices and concepts they were to employ within the current projects. A crash course in Agile and XP Programming was delivered during the first month of the course.

3.1. Our Flavor of Agile/XP

Groups tend to be 6 members in size giving us up to 8 projects. The class has two blocks of time, two hours in length each week for project development. The course spans 17 weeks in the fall term and 15 weeks in the spring term providing approximately 128 hours of designated class time for development. The students understand that time outside the designated class time will be required to accomplish typical projects. Groups must agree among themselves when and how much time in addition to class time will be spent on completing the project.

In order to use index cards as suggested by XP [9], we introduced removable corkboards to the labs. The boards are small (48”x36”) and mounted on brackets so that they can be easily removed when the labs are in use by other classes. These boards are used to display story and task cards that are relevant to the current release. During the work period, the students remove the task on which they are working; otherwise, the card is to be displayed. We take digital images of the task boards as an ongoing record of project progress.



Figure 1. Task board during release

In addition, the labs are fitted with whiteboards and data projectors that can be used as required. The students are encouraged to use the whiteboards to explore models and architecture. When they develop something they wish

to keep, we take a digital picture of the whiteboard; the picture becomes part of the documentation. There is also a small boardroom with whiteboard and network connection where the students may meet with their clients and make presentations.

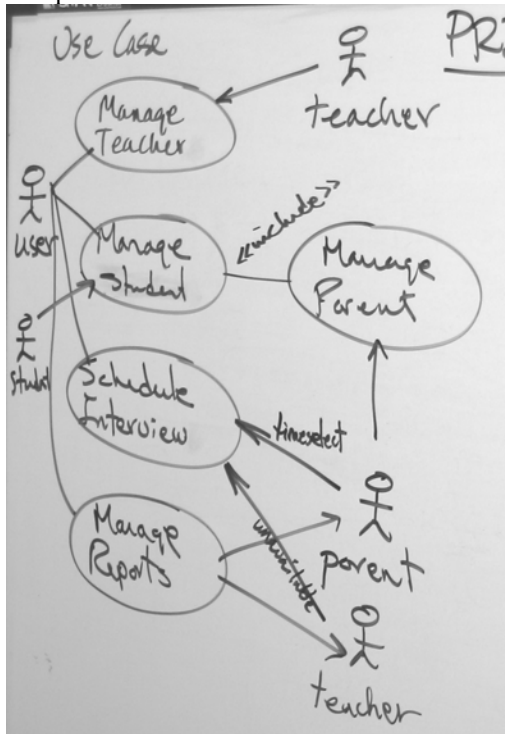


Figure 2. Whiteboard image of a model.

At the beginning of the class in the fall, we discussed with the students how we were going to manage the projects this year, employing iterations with one iteration per release [9]. The activities during our first release did not differ much from previous years. Students met with their clients and obtained a first description of the project. Students were expected to produce a Project Charter and a UML Use-Case model of the project along with a first set of stories by the end of the release that lasted approximately three weeks. The instructors then divided the remaining time in the term into fixed release blocks leading them to release often [2].

Each release follows a standard general format. In the first class period of the release, if a previous release was just completed, the students are expected to produce a retrospective [11] identifying what worked well and what did not during the release. Students are then to prepare a release plan for the current release that lists the stories to be implemented. Stories are broken down into tasks with an estimated completion time on each task card. All current release stories and tasks are to be displayed on their board.

Tasks include code development, model development and additional documentation. The incrementally

developed UML models that reflect the current state of the application under development form part of the deliverables for the release.

Each two-hour period begins with groups conducting a short standup meeting [10] to identify what will be accomplished during that work session.

Although Agile/XP requires clients to be on site [2, 11], our clients are usually not on site during development, unless the client is from another program within the institute. Thus most client contact is through email and phone calls. Meetings are limited to special occasions on or shortly after release dates. However, clients are advised that communication will be frequent and that we require quick responses to all inquiries.

Following XP pair programming, students are expected to work in pairs on all tasks. Each pair of students shares a computer workstation and workspace. The labs are arranged as a series of rows of tables across the room with up to four workstations on the row. Access to each row of tables is from the center of the room [12]. Other classes are held in these labs during the day. The labs are normally available for additional time after hours and on weekends.

As tasks are completed, the task card is checked off and the actual time recorded, as well as the names of the students who worked on the task. The back of the card and additional cards are used to record time if the task requires multiple two-hour sessions and XP pairs to complete.

To guard against the silo effect mentioned above, instructors are enforcing the co-ownership [10] of the project deliverables by making sure everyone works on all kinds of tasks at some point in the project development. This is monitored through analysis of the task cards. If someone is becoming entrenched in one kind of task it becomes quickly evident. The instructor/coach can then step in and reiterate the importance of working on all different tasks regardless of the strengths and weaknesses of the individual.

We encourage a spike [10] effort only when they truly need to experiment with an idea or component to see if it will be useful in completing the task at hand. Spikes are discouraged as a learning tool. When a student is blocked by how to go about doing something, we request that they quickly check with their group and then come see one of the instructors. Often the block is simply because they did not know how to proceed. If, however, they have a novel or nontraditional problem to address, they are encouraged to spike a possible solution.

Test-driven development (TDD) [13] is also encouraged and forms a small part of the evaluation structure. Because this is our first attempt at using an Agile/XP approach in this course, we have no formal infrastructure to support TDD. We demonstrated and encouraged the use of assert functions found in the

languages employed, to develop code through assertions. This works well with their knowledge of abstract data type definitions and the UML pre- and post-conditions specified on class methods. Other courses in the program have begun incorporating TDD into their material as a way of reinforcing the TDD aspect of the capstone project.

3.2. Classroom Management

Two instructors are assigned to the project course for the full year. This course represents only a portion of the teaching load for each instructor. Instructors are not considered part of the group and therefore do not contribute directly to completing tasks. Instructors act mostly as coaches and observers during the process. We will get involved only when we see a difficulty that needs addressing or when a student pair is having difficulty with a programming concept or language feature. Often issues dealing with things like environment will cause a difficulty of which the student is previously unaware or unprepared to deal with. The project coach or any instructor with knowledge in the area of concern is the first source of assistance to the student.

Attendance is taken as a formality at the start of each two-hour work period. Otherwise, instructors walk through the lab periodically to observe, but the students work largely on their own.

Instructors also attend meetings between the client and the student group where possible. Occasionally meetings are held at a time when an instructor cannot be present. In these cases, the group is expected to provide a full report of the meeting to the instructor. Where an instructor is in attendance, only a meeting summary is required to be provided to the client confirming the outcomes of the meeting.

3.3. Observed Student Behavior

Almost immediately it became apparent that student interest was higher this year than in previous years. Students began developing screens and functionality almost immediately. The requirement to maintain synchronicity between the developed portion of the application and UML models did not seem to be difficult since maintaining the model component was minimal and supported code and database table development. Because functionality was being placed into the product under the guidance of the client, students were less obsessed with getting screen layouts perfected. They seemed to realize that the screens would be refactored in the weeks ahead and that a less than perfect screen could be perfected once functionality was completed. It should be noted that at a career fair early in the fall term, a number of companies asked students about the Agile approach and whether they

were studying Agile concepts in their courses. This certainly added to the interest the students had in learning Agile.

Not all students quickly grasped the usefulness of the standup meeting. Some saw the meeting as a waste of time since they already knew what they were going to do during the work period. Over time it seemed to become clear to them that the meeting did provide a mechanism for knowing what everyone was going to do during the work period and what problems existed for others.

Creating and maintaining task cards seemed to be another concept that did not evidence its value initially. Students seemed to consider maintaining the task cards as an imposition and not of any real value. However, over time, the groups that did not embrace the task card concepts as readily, began to notice that the other groups got down to work quickly and seemed more focused and organized in their approach to the work of the day. By the beginning of the second term, all groups were aggressively managing their stories and task cards, using them as both a road map towards work still to do, and as a record of work accomplished. Students seemed to get a sense of satisfaction whenever they could add another completed task card to the pile. Students also quickly realized that task card time was an indicator of the amount of work they contributed to the project over the release. This was a valuable record of their individual contribution for later assessment.

A single strong personality within a group still creates difficulties [14]. However, by using the concept of co-ownership and selective pairing, we can address the difficulties of a single individual making all decisions and reduce any negative effects of such a personality [15]. The concept of co-ownership has established a shared responsibility to ensuring the success of the project. Students do not seem willing to wait as long to bring up an issue where one or more individuals are not co-operating with the rest of the group. Having co-ownership formally defined as a course parameter has empowered students who would not otherwise feel willing to bring up a difficulty and leads to resolution of such difficulties much earlier in the project. This, in turn, has had a two-fold effect: the stronger student has had to relinquish some tasks to others perhaps not as skilled in the task. Secondly, weaker students have had to address these tasks and have discovered that they can accomplish what is needed even though they lacked confidence initially.

The requirement to deliver often has had an interesting effect on when certain problems come to light. In previous years, the only delivery in most cases was at the end of the project. Whatever software had been developed was packaged and presented to the client. It would invariably be at this time that any problems with the installation, due to factors such as operating system incompatibilities, missing runtime libraries, missing

support software, and difficulties with connecting to servers (ex. making a connection through a firewall) would appear. Often there was not enough time for such difficulties to be overcome and the client would be left with an application that was not useable. In the current set of projects, the first release included first time installation of any support software. The problems mentioned above began appearing immediately with this release. Students were quick to realize that getting the runtime software correctly installed and configured was important. They would identify the various components and set up test machines with environments matching the client environment. Within a few releases, installation of most software went without error. Having these problems appear much earlier in the project also provided a good opportunity to discuss installation issues and ways to overcome difficulties. Thus students learned about and experienced first hand, encountering and solving installation problems.

In a number of our projects, the initial environment chosen for development was rejected in favor of another environment. The ability to change environments or languages while continuing to develop is very valuable. In previous years such a change would be impossible due to the lateness of source coding and the difficulties with “personal” ownership of code and the reluctance to discard already programmed functionality.

Often, knowledge about some concept, such as client-server interaction, does not really become important to the students until they have to utilize that knowledge (for example, to set up a client-server environment). In the Agile/XP approach, they are involved with applying such knowledge much earlier and over a longer period of time. Often their need for this knowledge precedes the presentations in other courses leading to pointed questions and motivation to learn more in those courses.

Retrospectives and frequent releases give students the opportunity to reassess their practices. For example, during the first term, two groups misunderstood the Agile/XP approach to be an unstructured, anything goes way of developing software. Repeatedly we emphasized that they were going to run into trouble if they did not become more disciplined. The first term ended with these two groups far behind the other groups in terms of progress. As a result both groups received a low mark. When the second term started, we reiterated that Agile/XP practices required discipline. Both groups began following the process more carefully and by the end of February, both groups were not only experiencing major successes but had outpaced the other groups in terms of project velocity.

4. Deliverables used for Student / Group Evaluation

Evaluation of projects in a course like ours is always problematic. First, not all projects are of equal complexity. Many project-specific issues impact the complexity of an application. Thus complexity must be considered when assigning marks at the end of the course. Under the Agile/XP approach, marks can be assigned based on releases. We can evaluate group and individual performance based on what was agreed to be released compared to what was actually released. The effects of complexity are diminished because the released functionality becomes the focus of evaluation and all groups tend to deliver a similar quantity of functionality. Because of the Agile/XP emphasis on testing functionality before including it in a release, the delivered functionality tends to be less error prone.

Evaluation of performance is based on the following criteria:

- Were all tasks chosen for the release actually delivered?
- Was functionality tested and useable by release time?
- Did each group member complete a similar amount of work based on his or her task cards?
- How well did estimates match actual times for task completion?
- Was everyone taking part in all group activities: client meetings, standup meetings, spikes and tasks?
- Did students work in XP pairs on all tasks?

Deliverables to the client include:

- Any new runtime software required to support the current functionality.
- Any support files of data.
- Database tables and data content necessary for the current release functionality.
- Updated UML models synchronized to the current completed functionality.
- Testing reports including unit and acceptance tests.
- User documentation for the functionality being released.
- Working, tested functionality as defined in the release plan, provided to the client at the beginning of the release period.

A number of interesting things come to light when the client is given a small set of functions that can be used. Assumptions appear about what was wanted or how the user will navigate a screen. Often the client discovers something that the students thought was wanted by the client but was not. After the release the client will come back with changes because it has become apparent that the way they thought they would do things is now obviously not the best approach. Through the process of multiple releases, the client and student group have the ability to tune the functionality to a more usable form.

Again this was not something possible in traditional approaches since actual use of the application only came at the end of the project.

5. Observed Effects of the Agile Approach on Project Activities and Outcomes

A number of benefits and some drawbacks are becoming evident.

5.1. Benefits

Positive aspects of Agile observed to date in the project course include:

- Encountering installation problems very early in the development process, often with the delivery of the first release, allows more time for resolution.
- There is better documentation of individual student work. Task cards record actual time spent by each student. It is a simple matter to tally the hours per student and then determine where time is being wasted because of other unproductive activities (i.e., web surfing, working on assignments from other classes).
- Dysfunctional groups become apparent much earlier. For example, in one group, cohesion problems were quite visible by the second release.
- Early coding has increased student interest in the project from the beginning.
- Early releases of partial functionality allow clients to better understand what is being created and make suggestions or changes while it is still feasible to modify the functionality.
- Coaches can obtain a better understand of the progress (velocity) of the project on a weekly basis.
- It is easier to bring the project back on track when a deviation is made due to misunderstanding of requirements, or due to a poor choice of solution for a particular problem.
- Releases contain both conceptual items, such as models and documentation, and items the client can begin using immediately. This makes it easier for the client to relate the functionality and the models.
- The visible progress apparent in every release seems to be a motivator for the students to undertake the next release with enthusiasm.
- Refactoring previous functionality is not seen as a difficulty because it is part of the process unlike in previous projects where any changes to working functionality were avoided.
- Task cards help focus the group on work to be completed; there is rarely a question from a group member such as "What should I do next?"
- Students have shown greater interest in their other courses since material in the other courses is often

immediately relevant to what they are doing in the project. Other instructors are stating that they are observing greater interest and participation over previous years.

- All students can observe the activity and progress of other groups. This also acts as a competitive motivation to continue to move forward.
- Students used to abandon quality in an effort to produce as much functionality as possible to meet their goal. Now, each release is a mini-project which is achievable. Any problems with released functionality can be refactored in the next released. With less functionality to produce, we have observed more use of error-handling and transaction control in the delivered code.

5.2. Drawbacks

Drawbacks we observed were:

- The Agile/XP approach can appear less disciplined to students. Students can become undisciplined if not closely monitored by their instructors to be following the Agile/XP approach carefully. Evaluation requires a component that measures how well the students follow the process. This acts as motivation to adhere to the Agile/XP approach during the early stages of the project.
- Clients are not present on site or during work periods, as they are supposed to be in an Agile/XP approach. We have experienced difficulty with getting timely answers to questions, or quick evaluations on releases. Periodically a group has had an unproductive work-period because a client response did not arrive on time. Better task identification and organization should overcome this in the future.
- Organizing the evaluation material and performing evaluations have been somewhat problematic; we are still learning all the possible ways in which evaluations can be performed.
- Due to the formal descriptions of the course, we are limited by how fully we can embrace Agile/XP; course outcomes must be met even if they do not blend well with Agile/XP.
- Due to the functional requirements of a release, the instructors may have to teach material scheduled to be taught in some other class at a later date. This tends to be offset by an increased student interest in that class when the material is presented.

6. Observed Effects of the Agile Process on Projects

The following observations have been made affecting the project deliverables:

- Early coding has given clients something concrete to understand and manipulate; this, in turn, has produced better feedback from clients and more focused effort on the part of students.
- Problems with requirements and solutions are encountered early in the development effort due to frequent releases; because releases are small, refactoring does not impose any major penalties to progress.
- Clients have an easier time understanding project scope and difficulties because of the frequent releases; they can quickly understand, when it becomes apparent something they asked for will not work as expected or is more problematic than it is functionally worth. This, in turn, changes the nature of the functionality to something more focused on what the client can use.
- Progress is more visible even if not significantly greater than in previous projects; this greater visibility acts as a self-motivator in later releases.
- Excessive documentation is avoided because documentation is driven by the development effort rather than being developed as a separate entity.
- At least one of our current clients became involved with our project course because we were using an Agile approach and they wished to observe how well it supported our efforts.

7. Conclusions

Overall, the employ of an Agile/XP approach in the project course has been largely a positive experience for both the instructors and the students. The approach has been more productive in terms of producing useable functionality for the client. The client has been able to direct the production of the functionality, the result being that the functionality delivered in a release was what the client requested for that release.

The approach has provided a better hands-on educational environment in which to learn and apply concepts. It has meshed neatly with other courses being delivered in parallel in that topics covered in other courses were timely and of immediate use by the students. This had the benefit of reinforcing the concepts and why they are important.

The students believe the use of an Agile/XP approach has proven to be a positive aspect of the program leading to future employment in industry. Clients and potential employers have shown interest in the Agile/XP approach reinforcing this belief that they are learning useful current technology.

We intend to continue to adapt and employ the Agile/XP approach in our project course in the future with expansion of the concepts and activities wherever appropriate. It is hoped that the experience we have had

with this first attempt at using Agile/XP within our project course will allow us to present a new version of the course for the calendar and get the course formally renamed with a new set of outcomes more focused on Agile/XP.

8. References

1. Manifesto for Agile Development, <http://www.agilemanifesto.org>.
2. K. Beck, "Embracing Change with Extreme Programming," *Computer*, IEEE, Vol.32, Issue No. 10, 1999, pp. 70-77.
3. <http://www.siastr.sk.ca/siastr/educationtraining/oncampusprograms/7266/5580/5850/index.shtml#courses>.
4. Schach, S.R., *Object-Oriented and Classical Software Engineering, Fifth Edition*, McGraw-Hill, 2002.
5. Ramakrishnan, R., and J. Gehrke, *Database Management Systems, Second Edition*, McGraw-Hill, 2000.
6. Fowler, M., *UML Distilled Third Edition, A Brief Guide to the Standard Object Modeling Language*, Pearson Education, Inc., 2004.
7. The History of FoxPro, http://www.foxprohistory.org/people_began.htm.
8. *Extreme Programming and Agile Methods – XP/Agile Universe 2004, 4th Conference on Extreme Programming and Agile Methods, Calgary, Canada, August 2004 Proceedings*, Springer-Verlag, 2004.
9. Iteration Planning, <http://www.extremeprogramming.org/rules/iterationplanning.html>.
10. Astels, D., G. Miller, and M. Novak, *A Practical Guide to eXtreme Programming*, Prentice Hall, 2002.
11. Principles behind the Agile Manifesto, <http://www.agilemanifesto.org/principles.html>.
12. Room 239A, <http://www.siastr.sk.ca/programsites/kelsey/cst/labs/239a.htm>, Room 239B, <http://www.siastr.sk.ca/programsites/kelsey/cst/labs/239b.htm>.
13. Beck, K., *Test Driven Development by Example*, Addison-Wesley, 2003.
14. G. Melnik and F. Maurer, "Perceptions of Agile Practices: A Student Survey," *Extreme Programming/Agile Universe*, Chicago, IL, 2002.
15. N. Katira, L. Williams and J. Osborne, "Towards Increasing the Compatibility of Student Pair Programmers," 27th International Conference on Software Engineering, St. Louis, MO, 2005.